SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – DATA INTERPRETATION

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Data Interpretation**
Weightage	**30% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* ***(BL3)***		
Student Name	**M.MANASA**	**Reg. No.**	**192425189**
Section/Batch	**Slot D/2024**	**Date**	**22-10-2026**

Code of Conduct:

I, M.MANASA(192425189), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: m.manasa_____

DATA INTERPRETATION

Question 21: HTTP Status Code Analysis

1. Question Statement

A web server records the following responses during one hour.

HTTP status code	Number of requests
200 OK	720
301 Moved Permanently	80
400 Bad Request	40
403 Forbidden	30
404 Not Found	100
500 Internal Server Error	30
Total	**1,000**

Table 1: Responses recorded by the web server in one hour

Tasks: (a) calculate the percentage of successful requests; (b) calculate the percentage of client-error responses (4xx); (c) identify the most frequent error status code and explain its meaning; (d) recommend two measures to reduce unsuccessful requests.

2. Introduction and Background

The World Wide Web works on a client–server model. A browser (the client) sends an **HTTP request** to a web server such as Apache, and the server returns an **HTTP response**. Every response begins with a three-digit **status code** which tells the client whether the request succeeded, was redirected, or failed. The first digit divides the codes into five classes:

Class	Meaning	Example in Table 1
1xx	Informational – request received, continuing	None recorded
2xx	Success – request understood and fulfilled	200 OK
3xx	Redirection – client must take further action	301 Moved Permanently
4xx	Client error – the request is wrong or not allowed	400, 403, 404
5xx	Server error – the server failed to handle a valid request	500 Internal Server Error

Table 2: HTTP status code classes

The codes in the question have the following meanings:

- **200 OK:** the request succeeded and the requested resource was returned.
- **301 Moved Permanently:** the resource now has a new URL; the browser is told to follow the new address.
- **400 Bad Request:** the server cannot understand the request because of malformed syntax or invalid data.
- **403 Forbidden:** the server understood the request but refuses to allow access (permission problem).
- **404 Not Found:** the server is working, but the requested URL does not exist.

- **500 Internal Server Error:** an unexpected fault (for example a PHP script error or a database failure) prevented the server from completing a valid request.

Analysing a server's status-code distribution is a basic and very practical form of data interpretation in web application deployment, because it quickly reveals broken links, access problems and application faults (BL3 – Apply).

The HTTP request–response cycle

To understand where each status code comes from, consider one complete exchange. The client opens a TCP connection to the server (port 80 for HTTP or 443 for HTTPS) and sends a request message made of a request line, headers and an optional body. The server processes it and replies with a status line, headers and the content.

Request (client to server)

```
GET /products.php?id=25 HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

Response when the page exists (200 OK)

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 5120

<html> ... page content ... </html>
```

Response when the page does not exist (404 Not Found)

```
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=UTF-8
Content-Length: 196
```

The first line of the response, *HTTP/1.1 404 Not Found*, is the status line. It is this line that the web server writes into its access log, and it is the data analysed in Table 1.

Where status codes are produced in a LAMP/WAMP stack

Component	Role	Status codes it typically causes
Apache (web server)	Receives the request, maps the URL to a file, applies access rules and redirects	301 (Redirect rules), 403 (permissions or Require rules), 404 (file missing), 400 (malformed request)
PHP (scripting)	Runs the application code for dynamic pages	500 (fatal script error, memory limit exceeded), 400 (invalid form or API input handled by the script)
MySQL (database)	Provides data to PHP	500 (connection refused, query failure, too many connections)
Operating system	File permissions and resources	403 (wrong owner or mode on files), 500 (out of memory or disk space)

Table 2a: Origin of status codes in a LAMP/WAMP stack

Where the access log is kept

On a LAMP server the access log is normally `/var/log/apache2/access.log` (Debian/Ubuntu) or `/var/log/httpd/access_log` (CentOS/RHEL), and on WAMP it is in the `apache/logs` folder. Each line records the client IP address, time, request, status code and response size. Counting the status-code field of these lines produces a table like Table 1.

```
awk '{print $9}' access.log | sort | uniq -c | sort -rn
```

3. Part (a): Percentage of Successful Requests

Assumption: a request is *successful* when the server returns a 2xx status code. In Table 1 the only 2xx code is 200 OK. The 301 response is a redirection, not a failure, and is discussed separately below.

Given: successful requests (200 OK) = 720; total requests = 1,000.

$$\textbf{Percentage of successful requests} = (720 \div 1{,}000) \times 100$$

$$= 0.72 \times 100 = 72\%$$

Answer: 72% of the requests were successful.

Interpretation

- Out of every 100 requests, 72 were served correctly. The remaining 28 requests (280 in total) were not a plain success.
- If the 301 redirects (8%) are also treated as working responses, because the browser reaches the new page, the effective "delivered" rate is (720 + 80) ÷ 1,000 = 80%. Redirects still cost an extra round trip, so they should be minimised.
- The true failure rate is the 4xx and 5xx responses: 170 + 30 = 200 requests = 20% of the traffic. This is high for a healthy site, where errors are normally kept in the low single digits.

The distribution of all responses is shown below.

Status code	Class	Requests	Share of total
200 OK	2xx Success	720	72.00%
301 Moved Permanently	3xx Redirect	80	8.00%
400 Bad Request	4xx Client error	40	4.00%
403 Forbidden	4xx Client error	30	3.00%
404 Not Found	4xx Client error	100	10.00%
500 Internal Server Error	5xx Server error	30	3.00%
Total		**1,000**	**100%**

Table 3: Share of each status code (Share = requests ÷ 1,000 × 100)

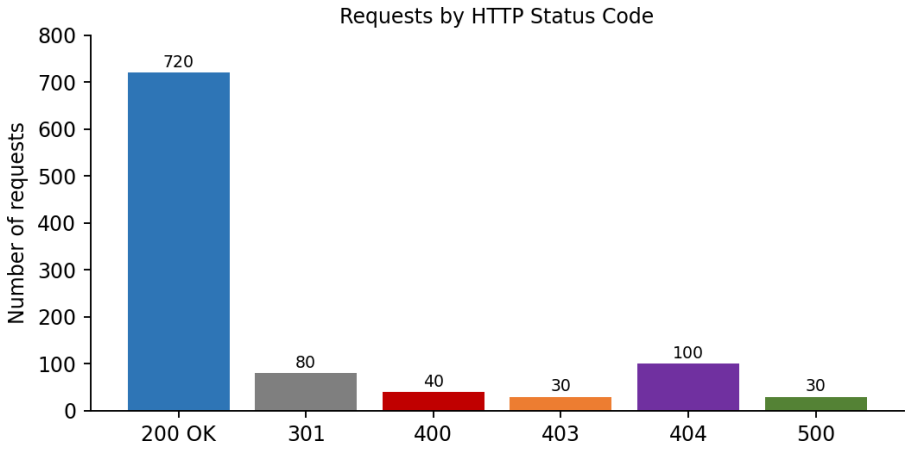


Figure 1: Requests by HTTP status code

Summary by status class

Class	Codes included	Requests	Percentage
Success (2xx)	200	720	72%
Redirection (3xx)	301	80	8%
Client error (4xx)	400, 403, 404	170	17%
Server error (5xx)	500	30	3%
Total		**1,000**	**100%**

Table 3a: Responses grouped by class (the percentages add up to 100%, which confirms the arithmetic)

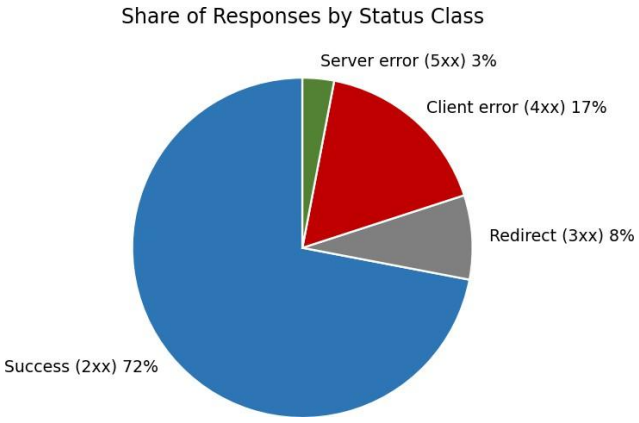


Figure 1a: Share of responses by status class

Ways of expressing the success rate

Measure	Formula	Value
Strict success rate (2xx only)	720 ÷ 1,000 × 100	72%
Delivered rate (2xx + 3xx)	(720 + 80) ÷ 1,000 × 100	80%
Error rate (4xx + 5xx)	(170 + 30) ÷ 1,000 × 100	20%
Unsuccessful rate (everything except 2xx)	(1,000 – 720) ÷ 1,000 × 100	28%

Table 3b: Alternative success and failure measures

The question asks for successful requests, so the strict figure of 72% is the main answer. The other measures show the whole picture: 72% + 28% = 100%, where the 28% is made up of 8% redirects and 20% errors.

Request rate

Over one hour (3,600 seconds) the server handled 1,000 requests, an average of 1,000 ÷ 60 ≈ 16.7 requests per minute, or about 0.28 requests per second. Within this, 720 ÷ 60 = 12 requests per minute were successful and 200 ÷ 60 ≈ 3.3 requests per minute failed with an error.

4. Part (b): Percentage of Client-Error Responses (4xx)

Given: 400 Bad Request = 40, 403 Forbidden = 30, 404 Not Found = 100.

Total 4xx responses = 40 + 30 + 100 = 170

Percentage of 4xx responses = (170 ÷ 1,000) × 100 = 17%

Answer: 17% of all responses were client errors.

Interpretation

- Client errors alone account for 17% of the traffic, more than twice the 8% taken by redirects and far above the 3% of server errors.
- Client errors make up 170 of the 200 failed responses, i.e. 170 ÷ 200 = 85% of all errors. The main cause of failed requests is therefore wrong, missing or restricted URLs rather than server faults.

4xx code	Requests	Share of all requests	Share of 4xx responses
400 Bad Request	40	4.00%	23.53%
403 Forbidden	30	3.00%	17.65%
404 Not Found	100	10.00%	58.82%
Total 4xx	**170**	**17.00%**	**100%**

Table 4: Breakdown of client-error responses (Share of 4xx = requests ÷ 170 × 100)

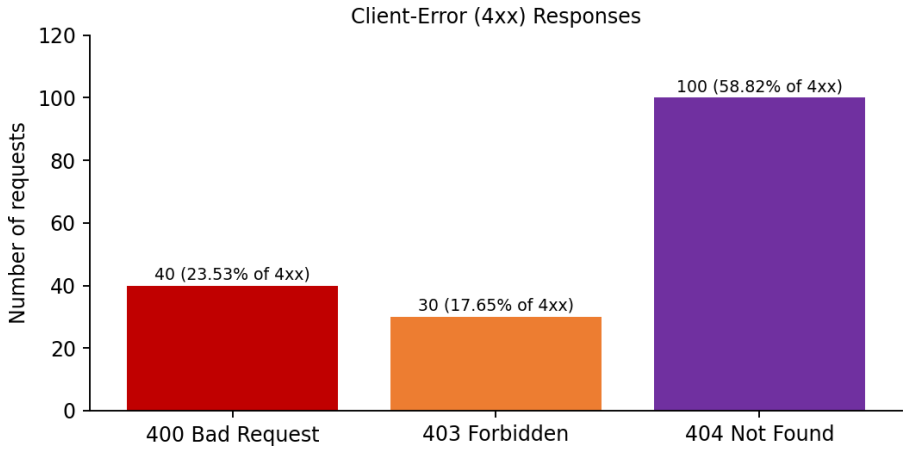


Figure 2: Client-error (4xx) responses

Further comparisons

- **4xx versus 5xx:** 170 ÷ 30 ≈ 5.7, so there are almost six client errors for every server error.
- **4xx versus redirects:** 170 ÷ 80 ≈ 2.1, so client errors occur about twice as often as 301 redirects.
- **Success versus 4xx:** 720 ÷ 170 ≈ 4.2, so roughly four successful responses are returned for every client error.
- **Rate:** 170 client errors in 60 minutes is about 2.8 per minute.

Understanding each client-error code

Code	Meaning	Typical causes	Who usually fixes it
400 Bad Request	Server cannot parse or accept the request	Malformed URL, invalid characters, oversized cookies or headers, corrupt form data	Developer (validation) and user (correct input)
403 Forbidden	Request understood but access refused	Wrong file permissions, directory listing disabled, IP blocked, login role not allowed	Administrator (configuration)
404 Not Found	No resource exists at the URL	Deleted or moved page, broken link, typing mistake, bot scanning	Developer / content owner

Table 4a: The three client-error codes in Table 1

Why a high 4xx share matters

A 4xx response is labelled a client error because the request itself is faulty or not permitted, but a website owner can still reduce most of them. Many 404 and 400 responses come from the site's own links and forms, so a high 4xx percentage usually points to maintenance problems on the website rather than to user mistakes.

5. Part (c): Most Frequent Error Status Code

Error codes are those in the 4xx and 5xx classes. Comparing them:

Error code	Requests	Rank	Share of all errors (200)
404 Not Found	100	1 (highest)	50.00%
400 Bad Request	40	2	20.00%
403 Forbidden	30	3 (tie)	15.00%
500 Internal Server Error	30	3 (tie)	15.00%

Table 5: Ranking of error status codes

Answer: 404 Not Found is the most frequent error, with 100 requests. It is 10% of all traffic and exactly half of all error responses, and it is more than double the next error (400 with 40 requests).

Meaning of 404 Not Found

A 404 response means that the web server was reached and is working, but it cannot find any resource at the requested URL. It is a client-side error code, but the cause can lie on either side.

Common causes

- Broken or outdated links inside the website, or links from other sites, pointing to pages that were deleted, renamed or moved.
- Mistyped URLs, wrong file names or case differences (on Linux/LAMP servers *Page.php* and *page.php* are different files).
- Missing images, CSS, JavaScript or favicon files referenced by a page.
- Automated bots and scanners probing for pages such as */admin* or */wp-login.php* that do not exist.

Impact

- Visitors see an error instead of content, which harms user experience and may lead them to leave the site.
- Search engines may lower the ranking of a site with many broken URLs.
- Every 404 still uses server CPU, bandwidth and log space without delivering any value.

Comparison with the 500 error

The 500 Internal Server Error (30 requests, 3%) is much less frequent, but it deserves special attention. It means that a *valid* request failed because of a fault on the server, such as a PHP fatal error or a database failure. Unlike a 404, it is never the visitor's fault, and a single bug can cause many 500 responses at once. Rank by frequency and rank by severity are therefore different: 404 is the most frequent error, while 500 is the most serious per occurrence.

Pareto view of the errors

Error code	Requests	Cumulative requests	Cumulative share of errors
404 Not Found	100	100	50%
400 Bad Request	40	140	70%
403 Forbidden	30	170	85%
500 Internal Server Error	30	200	100%

Table 5a: Cumulative error share. Fixing only 404 and 400 removes 70% of all errors.

6. Part (d): Two Measures to Reduce Unsuccessful Requests

The data show that 404 errors (10%) and other client errors dominate the failures, while 301 redirects (8%) add unnecessary requests and 400, 403 and 500 together add another 10%. The following two measures address these observations.

Measure 1: Fix broken links, update redirects and provide a helpful 404 page

This targets the 404 (10%) and 301 (8%) responses, which together are 18% of the traffic.

- Run a link checker or study the Apache access log to list the most requested missing URLs, for example: `grep " 404 " access.log | awk '{print $7}' | sort | uniq -c | sort -rn | head`.
- Correct internal links, or restore the missing pages and files.
- For pages that moved, add one 301 redirect from the old URL straight to the final URL, and update internal links so that they use the new URL directly and avoid redirect chains.
- Configure a custom error page with a search box and navigation links.

```
ErrorDocument 404 /errors/404.html
Redirect 301 /old-page.php https://example.com/new-page.php
```

Expected benefit: if about 70% of the 404 and 301 requests are removed, roughly 70 + 56 = 126 fewer unsuccessful or redirected requests per hour, and visitors reach the correct content in one step.

Measure 2: Validate requests, correct access permissions and monitor server errors

This targets the 400 (4%), 403 (3%) and 500 (3%) responses, which together are 10% of the traffic.

- **400 Bad Request:** add client-side and server-side input validation, make sure forms and API calls send correctly encoded URLs and parameters, and keep request headers and cookies within the server's size limits.
- **403 Forbidden:** review file and directory permissions and Apache *Require* rules so that legitimate users and public resources are accessible, while blocking real attackers or bots with a firewall or rate limiting (for example fail2ban or mod_evasive).
- **500 Internal Server Error:** check the Apache and PHP error logs, fix script errors, database connection failures and bad *.htaccess* rules, test changes in a staging environment before deployment, and enable monitoring and alerts for 5xx responses.

Expected benefit: fewer malformed and blocked requests, quicker detection and repair of server faults, and a more stable and secure application.

Before and after comparison (estimated)

The table gives an illustrative estimate of the effect of the two measures. The values are assumptions for explanation only, not measurements.

Metric	Current	After measures (estimate)	Change
Successful requests (200)	72%	≈ 85%	+ 13%
Client errors (4xx)	17%	≈ 6%	− 11%
Server errors (5xx)	3%	≈ 1%	− 2%
Redirects (301)	8%	≈ 3%	− 5%

Table 6: Estimated effect of the proposed improvements

Supporting measures

In addition to the two main measures, the following practices help to keep the number of unsuccessful requests low:

- **Enable caching and compression** (mod_expires, mod_deflate, browser cache headers) so that repeated requests for images, CSS and JavaScript are served from the cache and fewer requests reach the server.
- **Keep a sitemap and robots.txt** up to date so that search engines crawl only valid URLs and do not generate 404 requests for removed pages.
- **Use rate limiting and a firewall** to reduce scanner and bot traffic, which creates many 404 and 403 responses.
- **Automate log analysis** with a script or a tool such as GoAccess or the ELK stack, and set an alert when the 4xx or 5xx share crosses a threshold (for example 5%).
- **Test before deployment** with a link checker and automated tests in a staging environment so that broken URLs and script errors are caught before users see them.

Implementation plan

Priority	Action	Status code addressed	Effort
1	Fix or redirect the top missing URLs found in the log	404, 301	Low
2	Replace redirect chains and old internal links with final URLs	301	Low
3	Add input validation and correct URL encoding in forms and APIs	400	Medium
4	Review file permissions and Apache access rules	403	Low
5	Fix PHP/database faults found in error logs and add 5xx alerts	500	Medium
6	Add caching, rate limiting and scheduled log reports	All	Medium

Table 6a: Suggested order of work (highest impact first)

7. Impact on Server Performance and Users

Unsuccessful requests are not free. Even when the server returns only an error page, it must accept the connection, parse the request, search the file system, run any rewrite rules or scripts, build the response and write a log entry.

- **Wasted capacity:** 280 of the 1,000 requests (28%) did not deliver a normal page. If these were removed, the same server could carry about 39% more useful traffic for the same effort (1,000 ÷ 720 ≈ 1.39).
- **User experience:** visitors who meet a 404 or 500 page often leave. A site where 1 request in 5 ends in an error looks unreliable.
- **Search ranking and trust:** many broken URLs reduce crawl efficiency and credibility.
- **Security:** repeated 403 and 404 responses from one IP address often indicate scanning for vulnerabilities and should be monitored.
- **Operations:** large error volumes fill the logs and hide the real problems, making troubleshooting slower.

A practical target for a healthy website is a 2xx share above 95%, a 4xx share under 3% and a 5xx share under 0.5%. Against these targets the server in Table 1 needs improvement in all three measures.

8. Summary of Answers

Part	Question	Answer
(a)	Percentage of successful requests	72% (720 of 1,000)
(b)	Percentage of client-error (4xx) responses	17% (170 of 1,000)
(c)	Most frequent error status code	404 Not Found (100 requests), meaning the requested resource does not exist on the server
(d)	Two measures to reduce unsuccessful requests	1. Fix broken links, update redirects and add a custom 404 page. 2. Validate requests, correct permissions and monitor server errors.

Table 7: Final answers

9. Conclusion

The analysis of the one-hour server log shows that 72% of requests were served successfully, while 17% were client errors and 3% were server errors. 404 Not Found is the most frequent error with 100 requests, so broken or outdated URLs are the biggest single cause of unsuccessful requests. Repairing links and redirects, validating requests, correcting access permissions and monitoring the logs for server faults will increase the success rate, reduce wasted server resources and give users a more reliable web application.

10. Glossary of Key Terms

Term	Meaning
HTTP	HyperText Transfer Protocol, the application-layer protocol used for communication between web clients and servers
Status code	Three-digit number in the first line of an HTTP response that reports the result of the request
Redirect (301)	Response telling the client that a resource has permanently moved to a new URL
Client error (4xx)	Response saying that the request is invalid, unauthorised or refers to a missing resource
Server error (5xx)	Response saying that the server failed to fulfil a valid request
Access log	File in which the web server records every request together with its status code
Apache	Open-source web server used in LAMP and WAMP stacks
LAMP / WAMP	Linux (or Windows) + Apache + MySQL + PHP development and deployment stack
Redirect chain	Series of redirects (A to B to C) that wastes requests and time

Table 8: Glossary

Assessment Rubric

Criteria	Weight	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Calculation Accuracy	30%	Both percentages (72% and 17%) correct with complete working and the assumption stated.	Both correct with working, but the assumption is missing.	One correct, or minor arithmetic errors.	Incorrect or no working.	
Data Interpretation	20%	Identifies 404 as the most frequent error, compares all error codes and gives shares.	Identifies 404 with a basic comparison.	Identifies 404 with no reasoning.	Wrong code or no answer.	
Performance Recommendations	25%	Two relevant, practical, justified measures tied to the data, with implementation steps.	Two relevant measures with limited link to the data.	One valid measure, or two generic ones.	Irrelevant or missing.	
Technical Understanding of HTTP	15%	Clearly explains status-code classes and the request–response cycle.	Explains with minor gaps.	Partial or vague.	No understanding.	
Presentation & Organization	10%	Well structured with tables and headings, free of errors.	Well organized, minor issues.	Partially organized.	Poorly organized.	